

README

User Guide — Wiring CLI Coding Agents to Helix

Field	Value
Revision	2
Created	2026-09-05
Last modified	2026-09-07
Status	ACTIVE — the capability matrix in §3 is backed by end-to-end runs, not inference
Status summary	Documents how opencode, pi, crush, and Claude Code point at Helix's local LLM surfaces, which agent x surface combinations actually work (measured), and which two are broken and why.
Scope	Operator-facing usage of the four Helix LLM surfaces from four CLI coding agents on this host
Authority	Facts here are either the output of live probes (2026-09-05, see docs/research/cli_agent_config_schemas/EVIDENCE.md) or end-to-end agent runs (2026-09-07, see docs/qa/cli_agent_model_usability_20260907T113000Z/), or explicit vendor documentation, cited inline. Nothing here is asserted as "your current config" — verify with the commands given.

Anti-bluff note (revision 2). Revision 1 was written from endpoint probes and vendor docs without ever driving an agent end to end, and two of its conclusions turned out to be wrong when that was finally done — it recommended the surface that silently discards your request, and it called the only genuinely usable surface "not probed". Everything in §3 of this revision is

the recorded result of running the agent itself against the endpoint and checking the answer's *content*, not merely that a 200 came back. Where a combination has not been driven end to end, it is labelled UNTESTED, never "should work".

Table of contents

- 1. What gets wired, and where
- 2. The four Helix surfaces
- 3. Which agent x surface combinations actually work
- 4. Per-agent configuration
- 5. TLS for the self-signed gateway
- 6. Verifying each agent yourself
- 7. Undoing / removing the Helix providers
- 8. Re-running the installer: idempotency
- 9. The context-size cautionary tale
- 10. Where to go next

1. What gets wired, and where

Each supported CLI agent reads its own provider configuration from its own file, in its own format. Wiring Helix into an agent means adding one (or more) **custom OpenAI-compatible provider entries** pointing at one of the Helix LLM surfaces described in §2, plus the model id(s) that surface serves.

There is no single shared config file — four agents, four files, four shapes:

Agent	Version probed	Config file	Format
opencode	1.18.29	~/.config/ opencode/ opencode.json	JSON
pi	0.84.4	~/.pi/agent/ models.json	JSON
crush	0.91.2	~/.config/crush/ crushrc	Bash directives (not JSON)
	2.1.261	—	

Agent	Version probed	Config file	Format
claude (Claude Code)			N/A for local OpenAI-compatible providers; see §4.4

If an installer has run against your machine, you should find Helix entries already present in the files above for `opencode`, `pi`, and `crush`. If it has not run yet, §4 below shows you exactly what to add and how to add it by hand.

2. The four Helix surfaces

Helix exposes four distinct HTTP surfaces on this host. They are **not** interchangeable, and the differences that matter in practice are not the ones the model ids suggest:

Surface	URL	Auth	Model id(s) served	Usable context	Structured x
Coder (raw llama.cpp)	http://127.0.0.1:18434/v1	none	qwen2.5-coder-3b-instruct-q4_k_m	32,768 (n_ctx, = n_ctx_train; hard ceiling for this model)	emits a fe blob in the n ■
Gateway (HelixLLM)	https://127.0.0.1:8443/v1	none (self-signed TLS)	helixllm-anton-qwen2-5-coder-3b-instruct-q4_k_m-f6771589d190	32,768 (same llama.cpp process behind it)	real tool_ finish_reaso ■
HelixAgent	http://127.0.0.1:7061/v1	none	helixagent-llm, helixagent-debate, helixagent-ensemble,	≥ 200,046 measured	real tool_ (confirmed 2

Surface	URL	Auth	Model id(s) served	Usable context	Structured
HelixCode	http://127.0.0.1:8080	API key required (401 without one)	helix-llm, helix-debate /v1/messages (Anthropic-shaped) + /v1/chat/completions	not measured	not measured

Notes on the two measured columns, because both are load-bearing:

- **Usable context.** Coder/Gateway report `n_ctx`: 32768 in `/v1/models`, and that is the model's `n_ctx_train` — the maximum this model can be given, not a setting that can be raised. HelixAgent publishes no context figure at all, so it was measured directly: a 900,312-byte request was accepted and answered with `usage.prompt_tokens = 200046`, and the answer correctly recalled a marker planted at the **very front** of that prompt — so the surface neither refused the request nor silently truncated it.
- **Array-form message content.** Every current CLI agent may send a message whose content is a *list of parts* (`[{"type": "text", "text": "..."}]`) rather than a plain string. `pi` does this for every user turn; `opcode` does it unless `--pure` is passed; `crush` sends plain strings. Whether a surface understands that form decides whether your instruction reaches the model at all.

The Gateway silently discards array-form content

This is the single most dangerous behaviour on this host, because it produces a **200 OK with a fluent, plausible, completely unrelated answer** instead of an error. Measured 2026-09-07, identical request body, only the endpoint changed:

```
Coder    :18434  prompt_tokens=1480  ->
'HELIXOK'                                     ← correct
Gateway  :8443  prompt_tokens=577    -> 'Yes, I can help
with that. What would...' ← instruction gone
```

903 tokens — including the entire user instruction — were dropped on the floor, and nothing in the response says so. A smoke test that only checks for a non-error reply **passes** against

this. If you are using the Gateway from pi, or from opencode without --pure, you are not talking to the model you think you are.

The Gateway is safe for crush, which sends plain-string content.

3. Which agent x surface combinations actually work

Measured end to end on 2026-09-07 — each below means the agent itself was run against that endpoint and the **content** of its answer was checked, not merely that a response arrived. Raw figures in docs/qa/cli_agent_model_usability_20260907T113000Z/.

3.1 What each agent sends before you type anything

The reason most combinations fail has nothing to do with the model and everything to do with prompt size. On this host, **~950 agent skills are installed under ~/.claude/skills, ~/.config/crush/skills and ~/.pi/agent/skills, and each agent inlines all of them into its system prompt.** Measured with the Coder model's own tokenizer, before a single user turn is added:

Agent	Baseline prompt	Of which	Fits 32,768?
pi (default)	86,411 tokens	950 <skill> blocks, 4 tools	2.6× over
pi --no-skills	1,480 tokens	0 skills, 4 tools	
crush (default)	96,564 tokens	949 <skill> blocks, 26 tools	2.9× over
crush, no host skills	11,342 tokens	4 builtin skills, 26 tools	
opencode (default)	134,043 tokens	979 <skill> blocks, 120 tools (~40k tokens of tool schema alone)	4.1× over
opencode --pure	132,058 tokens	965 skills, 120 tools	4.0× over

This is why raising the Coder's `--ctx-size` from 4096 to 32768 did not, on its own, make these agents usable: 32,768 is this model's absolute maximum and every agent's baseline is two to four times larger.

3.2 The matrix

	Coder :18434	Gateway :8443	HelixAgent :7061
crush (default, full skills)	BLOCKED — request (96565 tokens) exceeds the available context size (32768 tokens)	BLOCKED — same 32,768 ceiling	WORKS, including real tool use
crush (host skills removed)	WORKS (11,342 tokens) — but no structured tool calls	UNTESTED end to end — crush's TLS trust of the self-signed cert did not complete within 110 s in this pass; curl --cacert to the same endpoint answers in 0.79 s, so this is a crush-side TLS issue, not a slow server	WORKS
pi (default, 950 skills)	BLOCKED — 86,411 > 32,768	BLOCKED — same ceiling	BLOCKED — array content → HTTP 400
pi --no-skills	WORKS (1,480 tokens) — chat only, tool calls arrive as fenced JSON text	BROKEN — 200 OK, instruction silently discarded (§2 warning)	BLOCKED — array content → HTTP 400
opencode (default)	BLOCKED — 134,043 > 32,768	BLOCKED — ceiling and silent-drop	BLOCKED — array content → HTTP 400
opencode --pure	BLOCKED — 132,058 > 32,768	BLOCKED — same ceiling	see §3.4

	Coder :18434	Gateway :8443	HelixAgent :7061
Claude Code	N/A — cannot consume an OpenAI-compatible provider (§4.4)	N/A	N/A

3.3 The recommendation, in one line

Use **crush against helixagent/helixagent-llm**. It is the only combination on this host that works today at the operator's real skill load, with real tool-calling, with nothing disabled:

```
$ crush run -m helixagent/helixagent-llm "Reply with exactly the word: BANANA7"
BANANA7
```

```
$ crush run -m helixagent/helixagent-llm \
  "Read the file probe_marker.txt in the current directory and
  reply with exactly its contents, nothing else."
I'll read the probe marker file.MARKER_FILE_CONTENT_ZQ7
```

Both runs above are against the **unmodified, already-installed** operator config — the second one drove crush's view tool through a real agentic loop. No installer change is needed for this; `install_agent_configs.sh` already registers helixagent for crush. What was missing was knowing to select it: revision 1 of this guide steered you to the Gateway and described HelixAgent as "not probed".

If you want pi, use `pi --no-skills` against `helixllm-coder`. It works, and it is genuinely limited: a 3B model with no structured tool-calling and no skills loaded.

3.4 opencode has no confirmed working path

opencode's baseline prompt is 134,043 tokens — four times the Coder/Gateway ceiling — and `--pure` only removes ~2,000 of them, because the weight is 979 inlined skills plus ~165 KB of tool schema for 120 tools, neither of which `--pure` touches. That rules out both llama.cpp surfaces outright.

HelixAgent has the capacity ($\geq 200,046$ tokens measured), and `opencode --pure` does send plain-string content, so it is the one combination that could work. It was attempted twice and produced **no verdict either way**: both runs were killed by their own timeout (300 s

and 500 s, SIGTERM, exit 143) with no output, and a request-capture proxy in front of the endpoint recorded *nothing* — so opencode never got as far as sending, and this is a client-side stall, not a rejection by HelixAgent (which answered a small request in 2.2 s while opencode was hung). **Treat it as UNTESTED**, and if you try it, judge it by whether the answer contains what you asked for, not by whether a response arrives.

3.5 The one defect that would unblock pi and opencode

Both are blocked on HelixAgent by a single, precisely-located fault: HelixAgent's OpenAI adapter cannot deserialize array-form content. Reproduce it in two commands — the only difference is the shape of content:

```
$ curl -sS -X POST http://127.0.0.1:7061/v1/chat/completions -H
'Content-Type: application/json' \
  -d '{"model":"helixagent-llm","messages":
[{"role":"user","content":"say OK"}], "max_tokens":8}'
{"...","content":"OK!    Let me know..."}
```

```
$ curl -sS -X POST http://127.0.0.1:7061/v1/chat/completions -H
'Content-Type: application/json' \
  -d '{"model":"helixagent-llm","messages":
[{"role":"user","content":[{"type":"text","text":"say
OK"}]}], "max_tokens":8}'
{"error":{"code":400,"message":"Invalid request format: json:
cannot unmarshal array into Go struct field
OpenAIMessage.messages.content of type
string","type":"invalid_request"}}
```

Both the Coder and the Gateway accept the second form (the Gateway accepts it and then discards it, which is worse). The fix lives in the `helix_agent` submodule's OpenAI request model, not in this repo's `scripts/` — it is recorded here rather than applied because it is a cross-repo change that needs its own tests and gates.

Fixing it is the highest-value change available: it would put the only long-context, tool-calling surface on this host within reach of pi and (capacity permitting) opencode, without disabling anything the operator has installed.

4. Per-agent configuration

4.1 opencode (1.18.29)

Config file: `~/.config/opencode/opencode.json`.

Before you wire this: opencode has no confirmed working path to any Helix surface on this host — its 134,043-token baseline prompt exceeds both llama.cpp surfaces, and it sends array-form content that HelixAgent rejects unless `--pure` is used. See §3.4. The shape below is correct; whether the combination *works* is §3's question.

Add a provider entry under `provider.<your-id>`, using the `@ai-sdk/openai-compatible` npm adapter, pointing `options.baseURL` at one of the surfaces from §2, and enumerating the model id(s) that surface serves. HelixAgent is shown here because it is the only surface with room for a real agent prompt (§3.1):

```
{
  "provider": {
    "helixagent": {
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseURL": "http://127.0.0.1:7061/v1"
      },
      "models": {
        "helixagent-llm": { "limit": { "context": 131072,
          "output": 4096 } }
      }
    }
  }
}
```

The `limit.context` of 131072 is a deliberately conservative figure: HelixAgent publishes no context value in `/v1/models`, and 200,046 prompt tokens were measured as accepted and un-truncated (§2). Do not copy 32768 here from the llama.cpp surfaces — that under-declares HelixAgent by about 4x and makes opencode compact or refuse work the endpoint would serve.

Notes:

- Omit `apiKey` entirely for these keyless local surfaces (Coder, Gateway, HelixAgent). Do not supply a placeholder value — the adapter treats an omitted key correctly for a server that does not check one.

- `opencode` **restarts required**: it reads this file at process start, so after adding or changing an entry, quit and relaunch `opencode` before it is visible.
- The **first use of a newly-declared `npm:-backed` provider** triggers `opencode` to download that `npm` package (and its dependencies) into its config directory — roughly 63 MB of `node_modules/` was observed for the adapter package plus its ecosystem. This needs network access the first time; subsequent runs reuse what was downloaded.
- Multiple config files under `~/.config/opencode/` (`config.json`, `opencode.json`, `opencode.jsonc`, and a TOML config) are all read and merged, later file wins. If you already have entries in more than one of these, know that they combine rather than one silently overriding the others outright — check all of them if something looks wrong.

4.2 pi (0.84.4)

Config file: `~/.pi/agent/models.json` (overridable via the `PI_CODING_AGENT_DIR` environment variable, which changes `pi`'s whole config directory, not just this file).

Before you wire this: `pi` sends array-form content on every user turn, which `HelixAgent` rejects (HTTP 400) and the Gateway silently discards. Its one working path on this host is the **Coder** surface with `--no-skills` (§3.2). Wiring `pi` to the Gateway will appear to work and will not be answering your question — see the warning in §2.

Add a provider under `providers.<your-id>`:

```
{
  "providers": {
    "helixllm-coder": {
      "baseUrl": "http://127.0.0.1:18434/v1",
      "api": "openai-completions",
      "apiKey": "local"
    }
  }
}
```

Notes:

- Unlike the other three agents, `pi` **requires a non-empty `apiKey` value even for a keyless server** — the field cannot be omitted. Any placeholder string (e.g. `"local"`) works; the value is not actually checked by these local surfaces.

- No restart is required: pi reloads provider config when you run its `/model` command inside a session.
- pi ships its own documentation under its installed package's `docs/` directory (`models.md`, `custom-provider.md`, `providers.md`, `settings.md`, `environment-variables.md`) — worth reading directly if you need an option not shown here.

4.3 crush (0.91.2)

Config file: `~/.config/crush/crushrc` — this is **Bash, not JSON**. Provider registration is a command line inside that file, or run directly against a running crush:

```
provider add helixgw --type openai-compat --base-url https://
127.0.0.1:8443/v1 --api-key local --discover-models true
```

Notes:

- With `--discover-models true`, crush queries the surface's `/v1/models` endpoint itself and populates the model list — you do not need to hand-declare model ids the way `opencode` and `pi` require. A bare `provider add` with no `model add` calls was enough, in testing, to make the served model visible in `crush models`.
- crush **restart required** after editing `crushrc` directly; re-running `provider add` against a live crush merges into the existing entry rather than duplicating it.
- TLS trust for the self-signed Gateway goes through the Go standard library's `SSL_CERT_FILE` environment variable — there is no TLS-related field anywhere in crush's own config schema (crush schema was inspected directly; no `tls/insecure/verify/cert/proxy` key exists). See §5.
- **crush writes a `.crush/` data directory into your current working directory by default.** If you run crush from inside this repository (or any project directory) without overriding this, it will create `.crush/` there. Pass `--cwd <dir>` and/or `--data-dir <dir>` to control where that lands, or see the FAQ entry in `FAQ.md` for how to avoid it entirely.

4.4 Claude Code (2.1.261)

Claude Code cannot accept a custom OpenAI-compatible provider.

This is a vendor constraint, not a Helix limitation: Anthropic's own documentation for the LLM Gateway Protocol lists exactly three client-selectable API formats for Claude Code — Anthropic Messages, Amazon Bedrock InvokeModel, and Google Agent Platform `rawPredict` — with

no OpenAI Chat Completions option at all, and states explicitly that "Anthropic doesn't endorse, maintain, or audit third-party gateway products, and doesn't support routing Claude Code to non-Claude models through any gateway." Claude Code's optional model-discovery feature also only keeps model ids that contain `claude` or `anthropic` (case-insensitively) anywhere in the string — a Helix model id would be filtered out even if discovery were pointed at a Helix surface.

The only surface with a plausible path into Claude Code is **HelixCode's Anthropic-shaped `/v1/messages` endpoint on `:8080`** (via `ANTHROPIC_BASE_URL` plus an API key, since `:8080` returns 401 without one). **This path was, at the time this guide was written, still under verification by a separate work stream — do not assume it works.** Check that stream's findings (or re-run the verification yourself against `:8080`) before relying on Claude Code against Helix for anything.

If you need Claude Code today, treat `opencode`, `pi`, or `crush` — pointed at Gateway or Coder — as the supported path for local Helix models.

5. TLS for the self-signed gateway

The Gateway surface (`https://127.0.0.1:8443/v1`) serves a self-signed TLS certificate. A plain HTTPS client will refuse it by default.

- `curl` needs `--cacert <path-to-cert>`, or it fails with `curl: (60) SSL certificate problem: self-signed certificate`.
- `crush` (Go binary) trusts the certificate when `SSL_CERT_FILE=<path-to-cert>` is set in its environment before it runs. This was confirmed directly: without it, `crush models` returns zero Gateway model entries (discovery silently fails); with it set, the Gateway's model id appears.
- `opencode` and Claude Code (both Node/Bun-based) recognize `NODE_EXTRA_CA_CERTS` and `NODE_TLS_REJECT_UNAUTHORIZED` as the equivalent knobs — these string literals are present in both shipped binaries, though a full end-to-end proof against the Gateway specifically was inconclusive for `opencode` (the test run hit a 280-second timeout with no output; see `docs/research/cli_agent_config_schemas/EVIDENCE.md` for the raw record — that is an honest "no verdict either way," not a documented failure).
- `pi` recognizes `SSL_CERT_FILE` in principle but a live end-to-end run against the Gateway also timed out (90 seconds) both with and

without `NODE_EXTRA_CA_CERTS` set — again, no usable verdict either way was obtained during this probing pass.

Practical takeaway: if you plan to use the Gateway surface, obtain its certificate and export `SSL_CERT_FILE` (or your agent's equivalent) before launching the agent. If model discovery for the Gateway silently returns nothing, TLS trust — not the provider config — is the first thing to check.

6. Verifying each agent yourself

Do not trust that wiring succeeded just because a file was written. There are **two** separate things to check, and passing the first tells you nothing about the second.

6.1 Is the provider registered?

```
opencode models | grep -i helix
pi --list-models | grep -i helix
crush models    | grep -i helix
```

A model id appearing here means the provider is *registered*. It does **not** mean a real request will succeed — see §3.

6.2 Does a real turn actually work?

Ask for a distinctive token and check that you get **exactly that token back**. This is the step that catches both failure modes on this host: the context-size rejection (loud) and the Gateway's silent content drop (quiet, and invisible to any check that only asks "did I get a reply?").

```
# crush — the recommended combination (§3.3)
crush run -m helixagent/helixagent-llm "Reply with exactly the
word: BANANA7"

# pi — needs --no-skills to fit the Coder's 32,768-token window
(§3.1)
pi -p --no-skills --provider helixllm-coder \
  --model qwen2.5-coder-3b-instruct-q4_k_m "Reply with exactly
the word: BANANA7"
```

Read the output, do not just check the exit code:

- BANANA7 — working.

- request (NNNNN tokens) exceeds the available context size (32768 tokens) — the agent's prompt does not fit that surface. See [§3.1](#).
- Any other fluent-but-unrelated answer ("Yes, I can help with that...") — you are almost certainly on the Gateway with an agent that sends array-form content, and your instruction was discarded. See the warning in [§2](#).
- Invalid request format: json: cannot unmarshal array into Go struct field OpenAIMessage.messages.content — HelixAgent with an agent that sends array-form content. See [§3.5](#).

To prove tool-calling end to end rather than just chat, make the agent *earn* the answer by reading something:

```
echo "MARKER_FILE_CONTENT_ZQ7" > /tmp/probe_marker.txt
cd /tmp && crush run -m helixagent/helixagent-llm \
  "Read the file probe_marker.txt in the current directory and
  reply with exactly its contents, nothing else."
```

If the marker comes back, the model called a tool, the tool ran, and its result reached the model. That is the bar — a chat reply alone does not clear it.

Claude Code is N/A for all of the above: it cannot consume a local OpenAI-compatible provider at all ([§4.4](#)).

7. Undoing / removing the Helix providers

Because each agent's Helix entry is just a provider block in that agent's own config file, removing Helix from an agent means removing that block and nothing else:

- **opencode:** delete the provider.<your-id> object from ~/.config/opencode/opencode.json (or whichever of the merged config files it lives in — see the note in [§4.1](#)), then restart opencode.
- **pi:** delete the providers.<your-id> object from ~/.pi/agent/models.json. No restart needed.
- **crush:** either delete the corresponding provider add ... line from crushrc, or run `crush model remove <provider>/<id>` for each declared model followed by removing the provider line, then restart crush.
- **Claude Code:** nothing to undo for the local OpenAI-compatible surfaces, since it never accepted them in the first place ([§4.4](#)). If a separate stream wired ANTHROPIC_BASE_URL / an API key for the

HelixCode surface, undo whatever mechanism that stream used to set those (environment variable, shell profile line, or settings file — check its own documentation).

None of this touches the Helix surfaces themselves (Coder, Gateway, HelixAgent, HelixCode keep running); it only removes an agent's knowledge of them.

8. Re-running the installer: idempotency

If an automated installer wires these providers for you, re-running it against an agent that already has a Helix entry should **merge into the existing config rather than duplicate or clobber it** — this is the expected, intended behavior for this kind of provider-registration step, consistent with how each agent's own tooling behaves when you register the same provider twice by hand (`crush provider add` on an existing id merges; re-declaring a model in `opencode.json` or `pi's models.json` simply overwrites that one entry, leaving everything else in the file untouched).

Concretely, a safe re-run should leave:

- any provider entries you added yourself, for services unrelated to Helix, completely untouched;
- the Helix entries either unchanged (if nothing changed on the Helix side) or updated in place (e.g. a new model id, a changed port) — never appended as a second, differently-named duplicate of the same surface.

Confirm this for your own installer rather than trusting this paragraph: run it twice in a row and diff the config file (or re-run the `grep -i helix` checks from §6 before and after) to see for yourself that nothing duplicated.

9. The context-size cautionary tale

Registering a model successfully does not mean it is usable. On 2026-09-05, with the Coder surface's context window (`n_ctx`) set to 4096 tokens, a real pi request against it was rejected outright:

```
400: {"code":400,"message":"request (141440 tokens) exceeds the available
```

```
context size (4096 tokens), try increasing
it", "n_prompt_tokens":141440, "n_ctx":4096}
```

The window was then raised 4096 → 32768. **That did not fix it**, and the shape of why is the lesson worth keeping:

- 32,768 is this model's `n_ctx_train` — its ceiling, not a conservative setting. There is no larger value available for it.
- Every mainstream agent's *baseline* prompt on this host is larger than that ceiling: pi 86,411, crush 96,564, opencode 134,043 tokens before a single user turn (§3.1).
- The weight is not the agent — it is **~950 skills installed under** `~/.claude/skills / ~/.config/crush/skills / ~/.pi/agent/skills`, which every one of these agents inlines into its system prompt. Remove them and pi drops to 1,480 tokens and crush to 11,342.

So there were only ever two real ways out, and both are now measured rather than guessed: **trim the prompt** (pi `--no-skills` → works against the Coder), or **use a surface with room** (crush → HelixAgent, ≥ 200,046 tokens measured → works at the full skill load, with tool use). Raising `--ctx-size` further was never one of them.

The generalisable rule: **a model appearing in <agent> models proves the plumbing works — it proves nothing about whether a real turn will fit, and nothing about whether the answer you get back is an answer to *your* question.** This host has one failure of each kind: the context rejection above, which is loud, and the Gateway's silent content drop (§2), which is not. See [FAQ.md](#) for the "lists the model but every request fails" question.

10. Where to go next

- [FAQ.md](#) — targeted answers to the questions this wiring tends to raise.
- [ARCHITECTURE.md](#) — a diagram of all four agents, all four surfaces, and which can reach which.
- `docs/research/cli_agent_config_schemas/EVIDENCE.md` — the raw, dated probe transcripts every factual claim in this guide traces back to.